

Termal

User Manual

Thomas Junier

September 4, 2026

Salmo trutta schuberti	T	E	R	M	-	-	-	A	L	-	-	-	-	-	-	-	-	-	-	-	-	-	-
Elephas roseus	T	E	R	M	i	n	A	L	-	A	L	I	G	N	M	-	T	V	I	E	W	E	R
Equus troianus	T	-	R	-	I	-	A	-	-	-	-	-	N	-	-	H	R	S	-	E	-	-	-
Buteo triplex	T	E	R	M	-	-	-	A	L	-	A	L	-	-	-	N	-	V	I	E	W	E	R
Oryctolagus paschalis	-	-	-	-	-	-	-	A	L	-	-	-	-	-	-	-	-	V	I	E	W	-	-
Anas laccata	-	-	-	-	-	-	-	A	L	-	-	-	-	-	-	-	-	-	-	-	-	-	-
Abies natalicia	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
Ciconia infantifera	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
Trifolium quadrifolium	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-

Contents

Overview	2
Basic usage	3
Starting the Program	3
Alignment File Formats	3
Help	3
Info mode	3
Options	3
Screen layout	3
Interaction	5
Prefix arguments	5
String Arguments	5
Navigation	5
Basics	5
Scrolling	5
Single-step scrolling	5
Scrolling by screenfuls	6
Aliases	6
Examples	6
Jumps	6
Position Jumps	6
Jumps to Search Matches	7
Jumps to Conserved Regions	7
Examples	7

Zooming	8
Zoomed-in Mode	8
Zoomed-out Modes	8
Searching	8
The Search Prefix Command	8
Searching Sequence Headers	8
Example:	9
Searching Sequences	9
Example:	9
Alignment Search	9
Example:	9
Setting the Reference	9
Example:	10
Sequence Metrics	10
Ordering the Sequences	10
Custom Ordering	10
Column Metrics	11
Display Settings	11
Resizing the Left Pane	11
Diff Mode	11
Residue Colormaps	11
Themes	12
Inverse Video	12
Modeline and feedback	12
Ex Commands (aka “:” commands)	12
:set	13
Limitations and Scope	13
Future Work	13
Design Philosophy	13
Companion Programs	14
termal-export	14
References	14

Overview

Termal is a terminal-based viewer for multiple sequence alignments (MSAs). It is designed for fast, keyboard-driven navigation of alignments of any size, particularly in remote or SSH-based environments where graphical tools are impractical.

Termal is a **read-only** viewer. It does not modify alignments. This may change in future versions.

Termal is presented in (Junier 2025).

A companion program, **termal-export**, produces SVG plots from alignments (see title page).

Basic usage

Starting the Program

Simply pass your alignment file as an argument to `termal`, e.g.:

```
termal my-alignment.msa
```

Alignment File Formats

Termal supports Fasta (default) and Stockholm (pass `-f stockholm` (or just `-f s`)) formats. Any alignment lines shorter than the longest one will be padded with space characters at the end.

Help

For general help, pass option `-h`; for a list of key bindings, pass `-b` or press `?` after launching `termal`.

Info mode

With option `-i/--info`, `termal` will output basic metrics about the alignment (such as number of sequences) and quit.

Options

The main options are shown in the table below (and are discussed in the text in the corresponding sections). Other options exist, but they are either experimental or used for debugging or testing. For a full list, do `termal -h`.

short	long	function
<code>-b</code>	<code>--show-bindings</code>	Show key bindings and exit successfully
<code>-i</code>	<code>--info</code>	Show alignment metrics and quit
	<code>--citation</code>	Show citation information and exit successfully
<code>-f</code>	<code>--format <FORMAT></code>	Sequence file format [<code>fasta/stockholm</code>] (or just <code>f/s</code>); default: <code>fasta</code>
<code>-o</code>	<code>--user-order <USER_ORDER></code>	User-supplied order (filename)
<code>-c</code>	<code>--color-map <COLOR_MAP></code>	Gecos color map (filename)
<code>-n</code>	<code>--dry-run</code>	Dry run - show parameters and quit
<code>-h</code>	<code>--help</code>	Print help
<code>-V</code>	<code>--version</code>	Show version

Screen layout

The interface is divided into four areas, starting from the top and left to right ([fig. 1](#)):

- **Headers pane** or simply **left pane**: shows sequence numbers and headers, as well as a barplot of the current [sequence metric](#)
- **Alignment pane** or **main pane**: shows the aligned sequences, some properties of the alignments, as well as some current UI settings.
- **Corner pane**: shows the current sequence metric and [ordering](#) (see below).
- **Reference pane** or **bottom pane**: shows horizontal position, the reference sequence (usually the consensus, but see [setting the reference](#)), and a conservation barplot

In addition, the last line contains a message area (“modeline”), which displays:

- pending command arguments (counts, searches), if any;

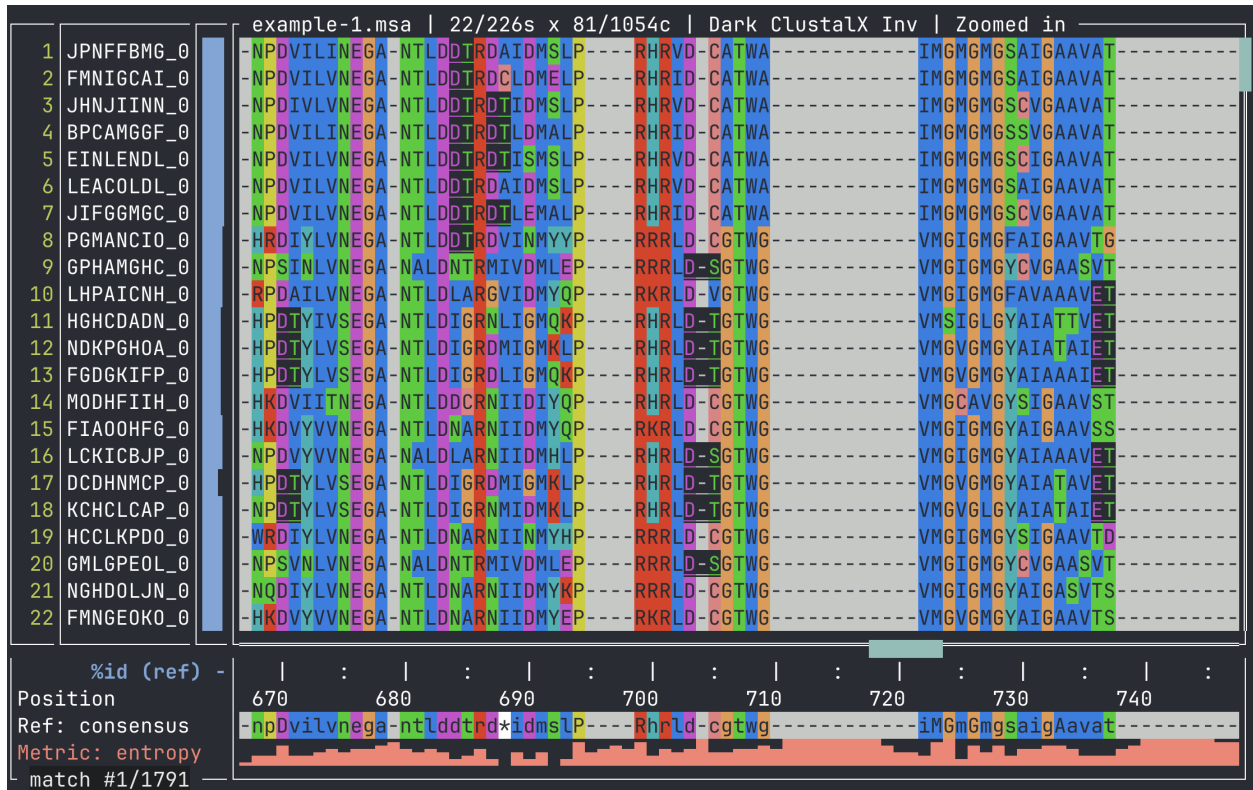


Figure 1: Terminal displaying example-1.msa. From left to right and top to bottom: headers pane, alignment pane, corner pane, reference pane, and modeline. The UI shows the result of a sequence search for [DE][ST], with matches highlighted in inverse video.

- search match information (when applicable).

Interaction

Termal is entirely keyboard-driven. Commands consist of one or two characters, optionally preceded by a numeric argument or followed by a string argument (see below).

Prefix arguments

Many commands (mostly motion) accept a *prefix argument*, which is an integer number typed *before* the command character. Typing any digit (except 0) starts a prefix argument. To cancel, press Esc. The meaning of the argument depends on the command. For example, scrolling commands such as j (“scroll one line down”) interpret a prefix argument as a repetition count: typing just j will move one sequence down, but typing 12j will move twelve sequences down.

If no prefix argument is given, commands default to 1.

String Arguments

String arguments are entered after typing the command character, and are entered by typing Return/Enter. Currently only the [header search](#) command (fh, shortcut "), [sequence search](#) command (fs, shortcut /), [alignment search](#) command (fa) as well as [reference selection](#) command (R) take a string argument.

Navigation

Basics

Navigation in Termal is inspired by [Vim](#) but simplified and adapted to multiple sequence alignments. The effect of motion commands depends on the [zoom mode](#), as follows:

- In zoomed-in mode, they change the portion of the alignment that is being displayed, much as a pager pages through a text file.
- In the zoomed-out modes, they move the zoom box.

Motion commands all take a prefix argument.

Scrolling

Scrolling commands move the visible part of the alignment by one or more steps. The prefix argument is interpreted as a repeat count.

Single-step scrolling

The following keys move the viewport by one step.

command	motion
[count]h	scroll <i>count</i> columns to the left
[count]l	scroll <i>count</i> columns to the right
[count]j	scroll <i>count</i> sequences down
[count]k	scroll <i>count</i> sequences up

Scrolling by screenfuls

To move by a screenful, use uppercase motion commands:

command	motion
[count]H	scroll <i>count</i> screenfuls to the left
[count]L	scroll <i>count</i> screenfuls to the right
[count]J	scroll <i>count</i> screenfuls down
[count]K	scroll <i>count</i> screenfuls up

Aliases

The arrow keys are aliases for the motion keys.

arrow	alias for
←	h
→	l
↓	j
↑	k
Shift-←	H
Shift-→	L
Shift-↓	J
Shift-↑	K

The Space key is an alias for J (move one screen down). This mimics the behaviour of less and other Unix pagers.

Examples

- h: move 1 column to the left
- 10l: move 10 columns to the right
- 5K: move 5 screens up
- 2<Spc>: move two screens down

Jumps

Position Jumps

Jump commands move directly to a position, specified by the prefix argument. The viewport moves so that the target sequence (respectively column) appears at the top (respectively left) of the alignment pane.

NOTE: in vertical jumps, the sequence's number is counted from the *first screen line*. This will be the same as the sequence's number in the alignment file, unless the sequences have been [reordered](#).

Absolute jumps In absolute jumps, the prefix argument denotes a specific sequence or column:

command	motion
[count]-	jump to sequence <i>count</i> (in the current order)
[count]=	jump to sequence <i>count</i> (in original file order)
[count]	jump to column <i>count</i>

Relative jumps In relative jumps, the prefix argument denotes a percentage of the alignment’s height or width, rounded to the nearest sequence/column.

command	motion
[count]%	jump to <i>count</i> % of the alignment’s height
[count]#	jump to <i>column</i> % of the alignment’s width

Jumps to Search Matches

Termal supports regular expression searches in both headers and sequences (see [Searching](#) for how to start a search). If any matches are found, Termal will automatically jump to the first match. To jump to the next or previous match, use `n` or `p`. The prefix argument allows to jump more than one match at a time. The Return key always jumps to the current match (this is useful if it is offscreen).

The sequence match jumps also have “vertical” variants, `N` and `P`. These also jump to the next (respectively, previous) match, but only if no horizontal motion is needed. This is useful for staying within a region of interest.

command	motion
[count] <code>n</code>	jump <i>count</i> matches forward
[count] <code>p</code>	jump <i>count</i> matches backward
[count] <code>N</code>	jump <i>count</i> matches forward, vertically
[count] <code>P</code>	jump <i>count</i> matches backward, vertically
<Return>	jump to the current match (which may be offscreen)

Next and previous match jumps wrap around, i.e. pressing `n` while on the last match will move back to the first one. If no matches were found, match jump commands have no effect.

By default, a jump will cause the viewport to move only if the next/previous match is off screen, avoiding repeated changes to the display. This can be changed by setting the `jump` variable to `center`, in which case the view is centered on the match at every jump. See [ex commands](#) for how to set variables.

Jumps to Conserved Regions

Termal defines a *conserved region* as either:

- a continuous stretch of positions where the [column metric](#) has a high value, or
- two conserved regions separated by only a few bases

where a “high” value is 0.2 or more, and “a few” bases means 3 or less. These thresholds can be changed by setting `lohi-threshold` and `lohi-gap`, respectively (see [ex commands](#)).

Examples

- `123|`: Jump to column 123 (if it exists).
- `200-:`: Jump to the 200th sequence from the top of the display, which may differ from the original file order if sequences have been reordered.
- `50%`: Jump to the vertical midpoint of the alignment.
- `25#`: Jump to one quarter of the alignment width.
- `n`: jump to the next match, if any
- `3p`: jump three matches backward, if possible
- `3N`: jump three matches forward (if possible), no horizontal motion
- `4(`: jump four conserved regions backward, if possible

Zooming

In Termal, *zooming* refers to changing how the alignment is displayed, rather than scaling characters graphically (although this can be done by changing the terminal emulator's font size). Zooming is only relevant if the whole alignment does not fit on screen. It is possible for the alignment to fit on the screen only in one of the two dimensions: in this case, the zooming only applies to the other dimension.

Zoomed-in Mode

In *zoomed-in* mode, Termal displays as many *adjacent* sequences and columns as will fit on the screen. This shows a detailed view of a portion of the alignment, but obscures its large-scale features. Motion commands change which portion of the alignment is displayed, causing new sequences/columns to appear into view and as many sequences/columns to disappear. Termal starts in zoomed-in mode.

Zoomed-out Modes

In *zoomed-out* mode, Termal shows the first and last sequence/column as well as a uniform sampling of sequences/columns between them. Enough sequences/columns are sampled to fill the screen; the other sequences/columns are not shown. This shows a “big picture” view of the alignment, at the expense of granularity. Motion commands affect the part of the alignment that will be shown when moving back to zoomed-in mode. This area is shown on screen as a rectangle known as the *zoom box*.

A variant of the *zoomed-out* mode works as above but preserves the alignment's aspect ratio. It is called *zoomed-out-AR*.

To cycle forward through the zoom modes, press `z`; to cycle backward, press `Z`.

Searching

The Search Prefix Command

All searches can be initiated via the `f` prefix command:

command	action
<code>fh</code>	header search (see Searching sequence headers)
<code>fs</code>	sequence search, ignoring gaps (see Searching sequences)
<code>fa</code>	alignment search: matches the raw alignment row, including gap characters

The standalone `"` and `/` commands are shortcuts for `fh` and `fs` respectively. `fa` (alignment search) is only available through the `f` prefix.

Searching Sequence Headers

Termal supports searching within sequence headers using regular expressions.

To start a search:

1. Press `"` (double quote)
2. Enter a [regular expression](#)
3. Press `<Enter>` to confirm

To cancel a search, press `<Esc>`.

During a search, the modeline displays:

- the index of the current match,
- the total number of matches.

Example:

```
"^Eco<Enter>
```

This jumps to the first sequence whose header starts with Eco (if any). The search order is according to the current [ordering](#).

Searching Sequences

Termal supports searching within sequences using regular expressions.

To start a search:

1. Press /
2. Enter a [regular expression](#)
3. Press <Enter> to confirm

To cancel a search, press <Esc>.

During a search, the modeline displays:

- the index of the current match,
- the total number of matches.

NOTE By default, sequence search ignores gap characters: the regular expression is matched against the ungapped sequence, and match positions are mapped back to the alignment. To search the raw alignment row including gaps (e.g. to match A-?T literally), use the alignment search command `fa` instead (see [Searching](#)).

Example:

```
/GAATTC<Enter>
```

This jumps to the first instance of GAATTC. The search order is according to the current [ordering](#), then left to right.

Alignment Search

Alignment search (`fa`) matches the raw alignment row, including gap characters. Use it when you want to explicitly match gaps in the pattern.

Example:

```
faA-*T<Enter>
```

This jumps to the first instance of A followed by zero or more gaps and then T in the alignment row. The search order is according to the current [ordering](#), then left to right.

Setting the Reference

Several features are computed with respect to a *reference sequence*: the [similarity metric](#) measures how similar each sequence is to the reference, and [diff mode](#) highlights residues that differ from it.

By default, the reference is the consensus sequence (which is automatically computed). However, the user may select any sequence in the alignment to serve as reference. This is done with the R command, which

takes a sequence number (with respect to the original file) as an argument. If the sequences are currently ordered according to the similarity metric, changing the reference triggers a recomputation of the ordering. Passing an empty argument reverts to the consensus.

Example:

R12<Enter>

This selects sequence #12 as reference. This is reflected in the corner pane, which now reads 'Ref: #12', and in the bottom pane, which displays sequence #12 instead of the consensus.

Sequence Metrics

The left pane displays a bar chart of the current *sequence metric*. This is a numeric property of the (aligned) sequences. Currently, there are two possible sequence metrics:

- a. Sequence **length** (not counting gaps)
- b. **Similarity** to the [reference](#)

To cycle forward through the metrics, press **t** (metric); press **T** to cycle backward. The current metric is displayed in the corner pane.

The sequences can be [ordered](#) according to the current metric.

Ordering the Sequences

Three orderings are available:

- **File order** (default): sequences appear as in the alignment file.
- **Metric order**: sequences are sorted by the current [sequence metric](#), ascending or descending.
- **Custom order**: supplied by the user via the **-o** option (see below).

To cycle forward through orderings, press **o**; to cycle backward, press **O**.

The current ordering is displayed in the corner pane:

symbol	meaning
-	file order
↑	current sequence metric, ascending
↓	current sequence metric, descending
u	user-supplied order (-o)

Custom Ordering

A custom ordering file can be supplied at launch:

```
termal -o my-order alignment.msa
```

An ordering file is simply the sequence headers in the desired order, one per line, *e.g.*:

```
AHMKMHDK_00298
CEFNMEKK_03699
IAGGDKJC_03995
FHL0DNDD_02091
HJACH0PP_04370
```

Note that headers must match those in the alignment file exactly.

Column Metrics

The bottom pane displays a barplot of the current *column metric*, a numeric property of each alignment column. Three metrics are available:

- Conservation:** High bar = well-conserved column; low bar = high variability. Derived from the column entropy.
- Coverage:** per-column fraction of non-gap residues.
- Weighted conservation:** The product of the previous two. The main difference with raw conservation is that a column may achieve high conservation even if it is mostly gaps, while its weighted conservation will not.

The metric is displayed in the *column metric bar plot*. To this end, they are normalized to fit the $[0, 1]$ interval. The (normalized) value of the column metric is used to define [conserved regions](#).

Display Settings

Resizing the Left Pane

The left pane can be widened (e.g. to show more of the sequence headers) with `>` and shrunk with `<`. This also resizes the corner pane. Both accept a prefix argument, which is by how many characters the pane is to be resized:

command	motion
<code>[count]></code>	widen the left pane by <i>count</i> characters
<code>[count]<</code>	shrink the left pane by <i>count</i> characters

Diff Mode

In diff mode, each residue in the alignment is displayed relative to the [reference](#):

symbol	meaning
letter	residue differs from the reference at this position
.	residue is identical to the reference
-	actual gap in the sequence

This makes it easy to spot variation at a glance, especially when combined with setting a specific sequence as reference.

To enter diff mode, press `dd`; to return to normal mode, press `D` or `dn`. The reference sequence row is always shown in full, regardless of diff mode.

Residue Colormaps

Termal supports four built-in residue color maps:

source	residue class	reference
ClustalX	amino acids	(Larkin et al. 2007)
Lesk	amino acids	(Lesk 2019)
JalView	nucleotides	(Waterhouse et al. 2009)
monochrome	both	themes

It is also possible to supply a custom color map:

```
termal -c colormap.json my-alignment.msa
```

where the colormap is a JSON file in the Gecos format (Kunzmann et al. (2020)). This is a straightforward format that looks like this:

```
{
  "name": "my-colormap",
  "alphabet": [
    "A",
    "C",
    "G",
    "T",
  ],
  "colors": {
    "A": "#71564e",
    "C": "#2a3d00",
    "G": "#004f7c",
    "T": "#b03f42",
  }
}
```

To cycle through colormaps, type `m` (forward) or `M` (backward). The initial colormap is ClustalX for amino acids or JalView for nucleotides, unless a custom colormap is supplied, in which case it becomes the initial colormap. The current colormap is displayed in the top border of the Termal screen.

Themes

Termal supports three themes: dark, light, and monochrome. To cycle through the themes, type `s` (forward) or `S` (backward). The current theme is displayed in the top border of the Termal screen.

NOTE Termal has been tested predominantly in a dark-themed terminal.

Inverse Video

By default, Termal displays the sequence residues in inverse video. To toggle to direct video (and back), press `i`. The current video mode is displayed in the top border of the Termal screen.

Modeline and feedback

The modeline (in the bottom border of the Termal screen) provides feedback about:

- pending numeric arguments,
- active search state,
- current search match index.

Ex Commands (aka “:” commands)

Ex commands¹ are used for the more rarely used operations. To enter an ex command, type `:`, then the command, then Return.

¹so named because they are accessed by typing `:`, like in the ex editor, and then in vi, Vim and friends

:set

The `:set` command is used to set the value of variables. Currently, the following variables are available:

Name	Value	Affects
<code>jump</code>	<code>lazy center</code>	Match jump behaviour
<code>lohi-threshold</code> <code>lt</code>	float, $\in [0, 1]$	Definition of conserved regions
<code>lohi-gap</code> <code>lg</code>	int, > 0	Definition of conserved regions

Note that some variables have short forms, *e.g.* `lohi-threshold` can be abbreviated to `lt`, etc.

Limitations and Scope

Termal currently does **not** support:

- editing alignments
- graphical export
- alignment formats other than Multi-fasta and Stockholm

Future Work

Features planned for the next release:

- a color map for vision-impaired users
- translation of aa regexps into nt, using IUPAC ambiguity codes - enabling search for amino acid patterns in DNA alignments

Features that will be added later

- `termal-export`, a companion program for producing publication-quality SVG figures directly from alignment files, is under active development and will be released separately. The figure in the title page of this manual provides an example of its output.
- showing fully conserved residues (à la EMBOSS's `showalign -show`)

Features under consideration

- showing translations of DNA sequences, where possible
- showing a phylogeny in the left pane
- simple edits (removing empty columns)
- saving parts of the alignments
- SAM/BAM pileups as a new “alignment” type (à la `samtools tview`).

Design Philosophy

Termal prioritizes:

- predictability over feature breadth,
- keyboard-driven navigation over menus,
- robustness over visual effects.

It is intended for users comfortable with terminal-based tools who value speed and clarity over graphical interaction.

Many commands, as well as the prefix argument syntax, were deliberately copied from [Vim](#).

Companion Programs

termal-export

termal-export is a companion program that generates publication-ready SVG figures from multiple sequence alignments. It supports selection of ranges of sequences and columns, and features the same color schemes as Termal. The SVG is designed to be easy to edit, e.g. the sequences and headers are grouped to allow easy selection of whole rows, etc.

Termal-export takes an alignment as argument, and produces SVG on standard output:

```
termal-export data/example-1.msa > example-1.svg.
```

For details, do

```
termal-export --help
```

References

- Junier, Thomas. 2025. "Termal: A Fast and Interactive Terminal-Based Viewer for Multiple Sequence Alignments." *Bioinformatics Advances*, September, vbaf208. <https://doi.org/10.1093/bioadv/vbaf208>.
- Kunzmann, P., B. E. Mayer, and K. Hamacher. 2020. "Substitution Matrix Based Color Schemes for Sequence Alignment Visualization." *BMC Bioinformatics* 21 (209). <https://doi.org/10.1186/s12859-020-3526-6>.
- Larkin, Mark A., Gordon Blackshields, Nigel P. Brown, et al. 2007. "Clustal w and Clustal x Version 2.0." *Bioinformatics* 23 (21): 2947–48. <https://doi.org/10.1093/bioinformatics/btm404>.
- Lesk, Arthur M. 2019. *Introduction to Bioinformatics*. 5th ed. Oxford University Press.
- Waterhouse, Andrew M., James B. Procter, David M. A. Martin, Michele Clamp, and Geoffrey J. Barton. 2009. "Jalview Version 2—a Multiple Sequence Alignment Editor and Analysis Workbench." *Bioinformatics* 25 (9): 1189–91. <https://doi.org/10.1093/bioinformatics/btp033>.